Lecture Notes: TwinCAT 3

Tc2_Utilities Library

March 17, 2026

Contents

Introduction **2**

1 Overview of Library Categories **2**

2 Memory Ring Buffer (FB_MemRingBuffer) **2**
- 2.1 Operating Logic . . . 2
- 2.2 Implementation Note . . . 3
- 2.3 Buffer Allocation . . . 3

3 Measuring Execution Time (PROFILER) **6**
- 3.1 Mechanism . . . 6
- 3.2 Outputs . . . 6
- 3.3 Utility . . . 6

4 Starting Windows Applications (NT_StartProcess) **9**
- 4.1 Key Inputs . . . 9
- 4.2 The "Response" Limitation . . . 9
- 4.3 Two-Stage Workaround . . . 9

5 Summary and Best Practices **13**
- 5.1 Self-Descriptive Naming . . . 13
- 5.2 Error Handling . . . 13
- 5.3 Exploration . . . 13

Quick Reference Card **13**

Introduction

The **Tc2_Utilities** library is a comprehensive "toolbox" for PLC developers, containing functionality that spans from operating system commands to specialized mathematical regulators. Unlike the standard library, this one is updated frequently with new tools to assist in daily software development.

> **Note:** The Tc2_Utilities library must be added as a reference to your PLC project. Right-click on "References" in your PLC project, select "Add library...", and search for "Tc2_Utilities".

1 Overview of Library Categories

The library is divided into several functional areas, including:

- **Operating System (OS):** Shutdowns, reboots, starting processes, and retrieving local system time.
- **STRING Functions:** Character conversion (upper/lower case) and substring searches.
- **System Functions:** Stopping/restarting TwinCAT, switching to Config Mode, and monitoring CPU usage.
- **Conversion Functions:** Tools for converting between various data types.
- **PLC Functions:** Handling persistent variables and measuring code execution.
- **Other:** PID regulators, real-time clocks, and memory buffers.

Category	Key Function Blocks	Purpose
OS Functions	NT_StartProcess, NT_Shutdown	Windows integration
System	TC_Restart, TC_Stop	TwinCAT control
Memory	FB_MemRingBuffer, FB_MemBufferSplit	Data buffering
Timing	PROFILER, NT_GetTime	Performance analysis
Conversion	LREAL_TO_STRING, STRING_TO_LREAL	Type conversion
Regulators	FB_CTRL_PID	Process control

Table 1: Tc2_Utilities Library Categories

2 Memory Ring Buffer (FB_MemRingBuffer)

This function block facilitates the storage of data records of varying lengths using the **FIFO (First In First Out)** principle.

2.1 Operating Logic

It uses read and write pointers. When data is added, the write pointer moves forward; when data is read, the read pointer follows. If the buffer reaches capacity and a new entry is added, the **oldest entry is automatically deleted** to make room.

2.2 Implementation Note

This block uses "Actions" (an older TwinCAT 2 convention) rather than modern Methods. For professional code, it is recommended to **wrap these actions** inside descriptive methods like `AddEvent()` or `GetAndRemove()` to improve readability.

2.3 Buffer Allocation

You must allocate a "chunk of memory" (an array of bytes) for the storage. Using the `SIZEOF` operator on your data structure allows you to calculate the exact buffer size needed for a specific number of entries.

Example: Buffer Size Calculation

```
// Data Unit Type (DUT): ST_Event
// Define a structure for the events we want to store in the buffer
// This structure can contain any data relevant to your application

TYPE ST_Event :
STRUCT
    // Timestamp when the event occurred
    tTimestamp       : DATE_AND_TIME;     // 4 bytes

    // Event identification
    nEventID         : UDINT;             // 4 bytes
    nSeverity        : INT;               // 2 bytes (1=Info, 2=Warning, 3=Error)

    // Event description
    sDescription     : STRING(80);        // 81 bytes (80 chars + null terminator)

    // Source information
    sSourceModule    : STRING(30);        // 31 bytes

    // Associated values
    fValue           : REAL;              // 4 bytes
    bAcknowledged    : BOOL;              // 1 byte

END_STRUCT
END_TYPE
// Total size: approximately 127 bytes per event
```

```
// PROGRAM: EventBufferDemo
// Demonstrates proper buffer size calculation using SIZEOF
// The buffer will store a fixed maximum number of events

PROGRAM EventBufferDemo
VAR CONSTANT
    // ====================================================
    // BUFFER SIZE CALCULATION
    // Use SIZEOF to get the exact size of your structure
    // This ensures the buffer is correctly sized regardless
    // of structure changes or compiler padding
    // ====================================================

    // Maximum number of events to store in the buffer
```

```
    MAX_EVENTS       : UDINT := 100;

    // Calculate the size of one event structure
    // SIZEOF returns the size in bytes
    EVENT_SIZE       : UDINT := SIZEOF(ST_Event);

    // Calculate total buffer size needed
    // Add extra bytes for internal management overhead
    // FB_MemRingBuffer needs ~8 bytes per entry for pointers
    OVERHEAD_PER_ENTRY : UDINT := 8;

    // Total buffer size calculation
    // (Structure size + overhead) * number of entries
    BUFFER_SIZE      : UDINT := (SIZEOF(ST_Event) + OVERHEAD_PER_ENTRY) * MAX_EVENTS;

    // Alternative: Simple calculation for demonstration
    // MAX_BUFFER_SIZE : UDINT := 20000;  // 20KB fixed buffer

END_VAR

VAR
    // ====================================================
    // BUFFER ARRAY ALLOCATION
    // The actual memory storage for the ring buffer
    // Must be an ARRAY OF BYTE with calculated size
    // ====================================================

    // Allocate buffer memory based on calculated size
    // Note: Array bounds are 0 to (SIZE-1)
    aBufferArray     : ARRAY[0..13499] OF BYTE;  // ~13.5KB for 100 events

    // For exact sizing, use a larger fixed array:
    // aBufferArray : ARRAY[0..19999] OF BYTE;  // 20KB buffer

    // ====================================================
    // RING BUFFER FUNCTION BLOCK INSTANCE
    // ====================================================
    fbEventBuffer    : FB_MemRingBuffer;

    // Event data for adding/reading
    stNewEvent       : ST_Event;          // Event to add
    stReadEvent      : ST_Event;          // Event read from buffer

    // Control and status
    bAddEvent        : BOOL;              // Trigger to add event
    bReadEvent       : BOOL;              // Trigger to read event
    bBufferOk        : BOOL;              // Operation success flag
    nEntriesInBuffer: UDINT;              // Current number of entries

    // Edge detection for triggers
    rtAddTrigger     : R_TRIG;
    rtReadTrigger    : R_TRIG;

END_VAR
```

```
// Buffer initialization and usage
```

```
// Initialize the ring buffer with our allocated memory
// This only needs to be done once (e.g., on first scan)
fbEventBuffer(
    pBuffer := ADR(aBufferArray),           // Pointer to buffer memory
    cbBuffer := SIZEOF(aBufferArray),       // Size of buffer in bytes
    bOverwrite := TRUE                      // Allow overwriting old entries
);

// Get current buffer statistics
nEntriesInBuffer := fbEventBuffer.nCount;

// ====================================================
// ADDING AN EVENT TO THE BUFFER
// Uses the A_AddTail action (FIFO - add to end)
// ====================================================
rtAddTrigger(CLK := bAddEvent);
IF rtAddTrigger.Q THEN
    // Prepare event data
    stNewEvent.tTimestamp := DT#2024-01-15-10:30:00;
    stNewEvent.nEventID := 1001;
    stNewEvent.nSeverity := 2;  // Warning
    stNewEvent.sDescription := 'Temperature exceeded threshold';
    stNewEvent.sSourceModule := 'TempController';
    stNewEvent.fValue := 85.5;
    stNewEvent.bAcknowledged := FALSE;

    // Add to buffer using A_AddTail action
    fbEventBuffer.A_AddTail(
        pWrite := ADR(stNewEvent),          // Pointer to data to add
        cbWrite := SIZEOF(stNewEvent)       // Size of data
    );
    bBufferOk := fbEventBuffer.bOk;
END_IF

// ====================================================
// READING AN EVENT FROM THE BUFFER
// Uses A_RemoveHead action (FIFO - read from beginning)
// ====================================================
rtReadTrigger(CLK := bReadEvent);
IF rtReadTrigger.Q THEN
    fbEventBuffer.A_RemoveHead(
        pRead := ADR(stReadEvent),          // Pointer to receive data
        cbRead := SIZEOF(stReadEvent)       // Size of data to read
    );
    bBufferOk := fbEventBuffer.bOk;

    // If bOk is TRUE, stReadEvent now contains the oldest event
    // If bOk is FALSE, the buffer was empty
END_IF
```

Ring Buffer Memory Layout						
Entry 0	Entry 1	Entry 2	···	Entry N-2	Entry N-1	
↑ Read Pointer				↑ Write Pointer		

Figure 1: FB_MemRingBuffer FIFO Operation

Warning: When using FB_MemRingBuffer with `bOverwrite := TRUE`, old data is silently discarded when the buffer is full. If you cannot afford to lose data, set `bOverwrite := FALSE` and monitor the `bOk` flag to detect when the buffer is full.

3 Measuring Execution Time (PROFILER)

The `PROFILER` function block is used to measure exactly how long a specific section of PLC code takes to execute in **microseconds**.

3.1 Mechanism

It is triggered by a **rising edge** on a `START` boolean to begin timing and a **falling edge** to stop.

3.2 Outputs

It provides a `PROFILERSTRUCT` containing the **minimum, maximum, and average** execution times based on the last 10 measurements.

3.3 Utility

This is essential for identifying CPU-intensive code and optimizing software performance.

Example: Using the PROFILER

```
// PROGRAM: PerformanceMonitor
// Demonstrates using PROFILER to measure code execution time
// Essential for identifying bottlenecks and optimizing performance

PROGRAM PerformanceMonitor
VAR
    // ====================================================
    // PROFILER FUNCTION BLOCK INSTANCE
    // Measures execution time in microseconds
    // ====================================================
    fbProfiler       : PROFILER;

    // Profiler results structure
    // Contains: tMin, tMax, tAvg (all in microseconds)
    stProfileResults: PROFILERSTRUCT;

    // Timing results for display/logging
    tMinTime         : TIME;              // Minimum execution time
    tMaxTime         : TIME;              // Maximum execution time
    tAvgTime         : TIME;              // Average execution time (last 10)
    nMeasurementCount : UDINT := 0;       // Number of measurements taken
```

```

    // Variables for intensive calculation demo
    nFactorialInput : INT := 12;        // Calculate factorial of this number
    nFactorialResult: DINT;             // Factorial result
    i               : INT;              // Loop counter

    // Array sorting demo (CPU-intensive)
    aSortArray      : ARRAY[0..999] OF INT;
    nTemp           : INT;
    j               : INT;
    bArraySorted    : BOOL := FALSE;

    // Control
    bRunMeasurement : BOOL := TRUE;     // Enable profiling
    bFirstScan      : BOOL := TRUE;     // First scan flag

END_VAR

// Initialize array with random-like values on first scan
IF bFirstScan THEN
    FOR i := 0 TO 999 DO
        aSortArray[i] := 1000 - i;      // Reverse sorted (worst case)
    END_FOR
    bFirstScan := FALSE;
END_IF

// ====================================================
// PROFILER USAGE - START TIMING
// Call with START := TRUE to begin measurement
// This captures the start timestamp
// ====================================================
fbProfiler(START := TRUE);

// ====================================================
// INTENSIVE CODE SECTION TO MEASURE
// Place the code you want to profile between
// the START := TRUE and START := FALSE calls
// ====================================================

// Example 1: Factorial calculation
nFactorialResult := 1;
FOR i := 1 TO nFactorialInput DO
    nFactorialResult := nFactorialResult * i;
END_FOR
// Result: 12! = 479001600

// Example 2: Simple bubble sort (intentionally slow for demo)
// This is CPU-intensive and good for profiling
IF bRunMeasurement AND NOT bArraySorted THEN
    FOR i := 0 TO 998 DO
        FOR j := 0 TO 998 - i DO
            IF aSortArray[j] > aSortArray[j + 1] THEN
                // Swap elements
                nTemp := aSortArray[j];
                aSortArray[j] := aSortArray[j + 1];
                aSortArray[j + 1] := nTemp;
```

```
            END_IF
        END_FOR
    END_FOR
    bArraySorted := TRUE;
END_IF

// Example 3: Mathematical operations
FOR i := 1 TO 100 DO
    // Trigonometric calculations (CPU-intensive)
    // fResult := SIN(INT_TO_REAL(i) * 0.01) * COS(INT_TO_REAL(i) *
   0.02);
END_FOR

// ====================================================
// PROFILER USAGE - STOP TIMING
// Call with START := FALSE to end measurement
// This calculates the elapsed time
// ====================================================
fbProfiler(START := FALSE);

// ====================================================
// RETRIEVE PROFILER RESULTS
// The PROFILERSTRUCT contains statistics from
// the last 10 measurements
// ====================================================
stProfileResults := fbProfiler.RESULT;

// Extract individual timing values
tMinTime := stProfileResults.tMin;      // Fastest execution
tMaxTime := stProfileResults.tMax;      // Slowest execution
tAvgTime := stProfileResults.tAvg;      // Average of last 10

// Increment measurement counter
nMeasurementCount := nMeasurementCount + 1;
```

```
// Extended example: Profiling multiple code sections

VAR
    fbProfiler1     : PROFILER;         // Profile section 1
    fbProfiler2     : PROFILER;         // Profile section 2
    fbProfiler3     : PROFILER;         // Profile section 3

    stResults1      : PROFILERSTRUCT;
    stResults2      : PROFILERSTRUCT;
    stResults3      : PROFILERSTRUCT;
END_VAR

// Profile Section 1: Data processing
fbProfiler1(START := TRUE);
    // ... data processing code ...
fbProfiler1(START := FALSE);
stResults1 := fbProfiler1.RESULT;

// Profile Section 2: Communication handling
fbProfiler2(START := TRUE);
    // ... communication code ...
fbProfiler2(START := FALSE);
stResults2 := fbProfiler2.RESULT;
```

```
// Profile Section 3: Motion calculations
fbProfiler3(START := TRUE);
    // ... motion calculations ...
fbProfiler3(START := FALSE);
stResults3 := fbProfiler3.RESULT;

// Now you can compare which section takes the most time
// and focus optimization efforts accordingly
```

Output	Type	Description
RESULT.tMin	TIME	Minimum execution time (last 10)
RESULT.tMax	TIME	Maximum execution time (last 10)
RESULT.tAvg	TIME	Average execution time (last 10)

Table 2: PROFILER Output Structure

Tip: Use the PROFILER during development to identify performance bottlenecks. Pay special attention to tMax—a high maximum time relative to average may indicate occasional slow paths in your code that could cause cycle overruns under certain conditions.

4 Starting Windows Applications (NT_StartProcess)

This block allows the PLC to trigger applications within the Windows operating system.

4.1 Key Inputs

It requires the `PATHSTR` (complete application path) and `COMNDLINE` (optional parameters). It can also target a remote PLC via its `NETID`.

4.2 The "Response" Limitation

A major limitation of `NT_StartProcess` is that it **cannot receive a response back** from the launched process.

4.3 Two-Stage Workaround

To get data back (e.g., checking if a network "ping" was successful), developers often use a **PowerShell script**. The PLC starts the script via `NT_StartProcess`, and the script then writes the results back to a PLC variable using **ADS**.

Example: Starting Windows Processes

```
// PROGRAM: WindowsIntegration
// Demonstrates NT_StartProcess to launch Windows applications
// Useful for system integration, reporting, and external tools

PROGRAM WindowsIntegration
VAR
    // ==================================================
    // NT_StartProcess FUNCTION BLOCK INSTANCE
```

```
    // Launches Windows applications from the PLC
    // ==================================================
    fbStartNotepad  : NT_StartProcess;
    fbStartCalc     : NT_StartProcess;
    fbStartScript   : NT_StartProcess;

    // Control inputs
    bOpenNotepad    : BOOL;             // Trigger to open Notepad
    bOpenCalculator : BOOL;             // Trigger to open Calculator
    bRunScript      : BOOL;             // Trigger to run PowerShell script

    // Edge detection (must use rising edge trigger)
    rtNotepadTrigger: R_TRIG;
    rtCalcTrigger   : R_TRIG;
    rtScriptTrigger : R_TRIG;

    // Status outputs
    bNotepadBusy    : BOOL;
    bNotepadError   : BOOL;
    nNotepadErrID   : UDINT;

    // Path configuration
    // Note: Use double backslashes or forward slashes in paths
    sNotepadPath    : T_MaxString := 'C:\Windows\System32\notepad.exe';
    sFilePath       : T_MaxString := 'C:\Data\Logs\production_log.txt';
    sCalcPath       : T_MaxString := 'C:\Windows\System32\calc.exe';
    sScriptPath     : T_MaxString := 'C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe';
    sScriptArgs     : T_MaxString := '-ExecutionPolicy Bypass -File "C:\Scripts\CheckNetwork.ps1"';

    // State machine for async operation
    nState          : INT := 0;

END_VAR

// ==================================================
// EXAMPLE 1: Open Notepad with a specific file
// ==================================================
rtNotepadTrigger(CLK := bOpenNotepad);

CASE nState OF
    0:  // Idle - waiting for trigger
        IF rtNotepadTrigger.Q THEN
            nState := 1;
        END_IF

    1:  // Start the process
        fbStartNotepad(
            NETID       := '',                  // Local machine (empty = local)
            PATHSTR     := sNotepadPath,        // Application path
            DIRNAME     := 'C:\Data\Logs',      // Working directory
            COMNDLINE   := sFilePath,           // Command line: file to open
```

```
            START        := TRUE                    // Execute on rising edge
        );
        nState := 2;

    2:  // Wait for completion
        fbStartNotepad(START := FALSE);             // Reset START

        IF NOT fbStartNotepad.BUSY THEN
            bNotepadBusy := FALSE;
            bNotepadError := fbStartNotepad.ERR;
            nNotepadErrID := fbStartNotepad.ERRID;

            IF fbStartNotepad.ERR THEN
                // Handle error - log it, set alarm, etc.
                // Common errors:
                // - File not found
                // - Access denied
                // - Invalid path
            END_IF

            nState := 0;                            // Return to idle
        ELSE
            bNotepadBusy := TRUE;
        END_IF

END_CASE

// ====================================================
// EXAMPLE 2: Open Calculator (simple, no parameters)
// ====================================================
rtCalcTrigger(CLK := bOpenCalculator);

fbStartCalc(
    NETID       := '',
    PATHSTR     := sCalcPath,
    DIRNAME     := '',
    COMNDLINE   := '',                              // No command line arguments
    START       := rtCalcTrigger.Q                  // Trigger on rising edge
);

// ====================================================
// EXAMPLE 3: Run PowerShell script
// Useful for tasks that need to return data via ADS
// ====================================================
rtScriptTrigger(CLK := bRunScript);

fbStartScript(
    NETID       := '',
    PATHSTR     := sScriptPath,                     // PowerShell executable
    DIRNAME     := 'C:\Scripts',                    // Script directory
    COMNDLINE   := sScriptArgs,                     // Script path and arguments
    START       := rtScriptTrigger.Q
);
```

```
// Additional examples: Various Windows integrations

VAR
    fbStartExplorer : NT_StartProcess;
    fbStartBrowser  : NT_StartProcess;
    fbStartBackup   : NT_StartProcess;

    sExplorerPath   : T_MaxString := 'C:\Windows\explorer.exe';
    sFolderToOpen   : T_MaxString := 'C:\ProductionData\Reports';

    sBrowserPath    : T_MaxString := 'C:\Program Files\Mozilla Firefox\firefox.exe';
    sWebPage        : T_MaxString := 'http://192.168.1.100/status';

    sRobocopyPath   : T_MaxString := 'C:\Windows\System32\robocopy.exe';
    sBackupArgs     : T_MaxString := 'C:\Data D:\Backup\Data /MIR /LOG:C:\Logs\backup.log';
END_VAR

// Open Windows Explorer to a specific folder
fbStartExplorer(
    PATHSTR     := sExplorerPath,
    COMNDLINE   := sFolderToOpen,
    START       := bOpenFolder
);

// Open web browser to machine status page
fbStartBrowser(
    PATHSTR     := sBrowserPath,
    COMNDLINE   := sWebPage,
    START       := bOpenStatus
);

// Run backup using robocopy (file synchronization)
fbStartBackup(
    PATHSTR     := sRobocopyPath,
    COMNDLINE   := sBackupArgs,
    START       := bRunBackup
);
```

Input	Type	Description
NETID	T_AmsNetId	Target system (empty = local)
PATHSTR	T_MaxString	Full path to executable
DIRNAME	T_MaxString	Working directory
COMNDLINE	T_MaxString	Command line arguments
START	BOOL	Rising edge triggers execution

Table 3: NT_StartProcess Input Parameters

Warning: NT_StartProcess runs applications in the Windows user space, which is not real-time. There is no guarantee when the application will start or how long it will take.

Never use this for time-critical operations. Also, the PLC cannot receive any response from the launched application directly.

Note: To get responses from external processes, create a PowerShell or batch script that performs the operation and writes results back to the PLC using ADS (e.g., using the TwinCAT ADS .NET library or command-line tools).

5 Summary and Best Practices

5.1 Self-Descriptive Naming

When wrapping library functions, use names that reflect the actual action (e.g., `ReadAndRemove` instead of `RemoveHead`).

5.2 Error Handling

Always check the `bOk` or `bError` outputs of these blocks, especially when working with buffers that might be full or empty.

5.3 Exploration

Because the utilities library is so vast, it is highly encouraged to browse the official documentation to find specialized tools for specific problems.

Best Practice	Rationale
Use SIZEOF for buffer calculations	Adapts to structure changes automatically
Check bOk/bError after operations	Prevents silent failures
Use R_TRIG for START signals	Ensures single execution per trigger
Wrap actions in descriptive methods	Improves code readability
Profile critical code sections	Identifies optimization opportunities

Table 4: Tc2_Utilities Best Practices

Tip: The Tc2_Utilities library is extensively documented in the Beckhoff Information System. When facing a specific challenge, search the documentation—there's often a pre-built function block that solves your problem more elegantly than a custom solution.

Quick Reference Card

FB_MemRingBuffer

```
// Declaration
fbBuffer : FB_MemRingBuffer;
aMemory : ARRAY[0..9999] OF BYTE;

// Initialize
fbBuffer(pBuffer := ADR(aMemory), cbBuffer := SIZEOF(aMemory));
```

```

// Add data (FIFO)
fbBuffer.A_AddTail(pWrite := ADR(stData), cbWrite := SIZEOF(stData));

// Read and remove data (FIFO)
fbBuffer.A_RemoveHead(pRead := ADR(stData), cbRead := SIZEOF(stData))
    ;
```

PROFILER

```
// Declaration
fbProfiler : PROFILER;
stResults : PROFILERSTRUCT;

// Usage
fbProfiler(START := TRUE);
    // Code to measure
fbProfiler(START := FALSE);
stResults := fbProfiler.RESULT;
// stResults.tMin, .tMax, .tAvg contain timing data
```

NT_StartProcess

```
// Declaration
fbStart : NT_StartProcess;

// Usage (trigger on rising edge)
fbStart(
    PATHSTR := 'C:\Windows\notepad.exe',
    COMNDLINE := 'C:\file.txt',
    START := bTrigger
);
```

Common Error Codes

Error ID	Hex	Description
0	0x0000	No error
1	0x0001	Internal error
2	0x0002	Invalid parameter
6	0x0006	Target port not found
1793	0x0701	Buffer full
1794	0x0702	Buffer empty

Last Updated: March 17, 2026